A PROJECT REPORT

ON

LOCALMIND

Local-First Agentic Knowledge Copilot with a Custom MCP Memory Server

A project report submitted in partial fulfillment of the requirements
for the Bachelor of Technology programme

BACHELOR OF TECHNOLOGY
Minors in AIML

Submitted By

Trijal Burman — 23070123145

Tiya Singh — 23070123144

Gayatri Rajesh Apotikar — 23070123055

Nimita Jestin — 23070122259

UNDER THE GUIDANCE OF

Dr. Sonali Kothari



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Pune – 412115, Maharashtra State, India

https://www.sitpune.edu.in/

DEPARTMENT: MINORS IN AIML

ACADEMIC YEAR 23–27

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Pune – 412115, Maharashtra State, India
https://www.sitpune.edu.in/

DEPARTMENT: MINORS IN AIML

CERTIFICATE

This is to certify that the project work entitled “LocalMind: Local-First Agentic Knowledge Copilot with a Custom MCP Memory Server” has been prepared by Trijal Burman, Tiya Singh, Gayatri Rajesh Apotikar and Nimita Jestin, in partial fulfillment of the requirements of the Bachelor of Technology programme, Minors in AIML, at Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune, for the academic year 23–27.

Faculty certification and approval are to be completed after institutional review.

Guide	**Co-Guide (if applicable)**	**Head of Department**
Dr. Sonali Kothari	_____	_____
Signature: _____	Signature: _____	Signature: _____

Date: _____

DECLARATION

We declare that the project entitled “LocalMind: Local-First Agentic Knowledge Copilot with a Custom MCP Memory Server”, submitted to Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune, is our project work for the Bachelor of Technology programme, Minors in AIML. We are responsible for the claims, sources, evaluation records and final submission contained in this report.

Prior research and software sources are acknowledged in the references. AI assistance in software development, experimental support and report drafting is disclosed in Appendix C. We will review this declaration against institutional requirements before signing. No faculty approval, publication or plagiarism score is implied by this unsigned copy.

Name	**PRN**
Trijal Burman	23070123145
Tiya Singh	23070123144
Gayatri Rajesh Apotikar	23070123055
Nimita Jestin	23070122259

Department: Minors in AIML
Academic year: 23–27

Student signatures:
Trijal: _____ Tiya: _____
Gayatri: _____ Nimita: _____

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Sonali Kothari, for her guidance and supervision of LocalMind. Her support helped us frame the project around practical local knowledge management and the need to evaluate an AI system beyond the fluency of its responses.

We thank Symbiosis Institute of Technology, Pune, for the opportunity to undertake this project as part of Minors in AIML. We acknowledge the researchers and maintainers whose openly available work on retrieval, local inference, SQLite and the Model Context Protocol made the implementation and study possible.

We also thank our families, classmates and friends for their support. We acknowledge the AI-assisted development and writing process and retain responsibility for checking every submitted claim. In particular, we preserve the unsuccessful experimental outcomes because they are essential to understanding the system and improving it responsibly.

Trijal Burman
Tiya Singh
Gayatri Rajesh Apotikar
Nimita Jestin

ABSTRACT

LocalMind is a local-first knowledge copilot that combines approved-folder indexing, persistent project memory, local language-model inference and a custom Model Context Protocol (MCP) server. The system uses a browser interface, FastAPI, SQLite FTS5, dense cosine retrieval, reciprocal rank fusion and Ollama. It supports source inspection, conversational assistance, document summarization and saved project plans without mandatory paid inference APIs after installation.

Evaluation on LM-Synth v1 uses 39 markdown files, 141 chunks and 96 retrieval questions. Historical lexical and hybrid retrieval achieve complete evidence coverage at five candidates on 96/96 questions; dense retrieval achieves 91/96. The original generation paths answer 0/24 positive questions completely after losing the reference block. Following an evaluation-only diagnostic, the production fitter was repaired to measure actual serialized cost. A direct replay retains reference evidence in 64/64 requests and yields 7/24 complete answers for RAG and 3/24 for the MCP agent. Historical summaries retain 10/24 requested facts and were not rerun.

The locally applied repair passes 62 Linux tests and 99 tests in the Windows source copy on Linux, with six native Windows checks skipped. The replay reuses inspected questions and is not independent accuracy confirmation. The study demonstrates feasible local integration on a Ryzen 7 250 / RTX 5050 laptop while separating context delivery from remaining passage-selection and generation limits.

Keywords: local-first AI; MCP; persistent memory; agentic RAG; SQLite; evidence delivery.

TABLE OF CONTENTS

Contents	**Page**
Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v–vi
List of Tables	vii
List of Figures	viii
List of Abbreviations	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem statement	2
1.3 Scope of research / 1.4 Research hypothesis	3
1.5 Objectives	4
1.6 Organization of the report	5
CHAPTER 2: LITERATURE REVIEW	6
2.1 Background	6–11
2.2 Literature summary and research gap	12
CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION	13
3.1 Software tools / frameworks	13
3.2 Hardware tools	14
3.3 Work Breakdown Structure	15
3.4 Functional Requirements	16–17
3.5 Non Functional Requirements	18
3.6 Project Cost Estimation	19

TABLE OF CONTENTS — continued

Contents	Page
CHAPTER 4: METHODOLOGY	20
4.1 System Architecture and Data Model	20–23
4.2 Ingestion and Hybrid Retrieval	24–28
4.3 MCP and Bounded Agent Execution	29–31
4.4 Privacy Controls and Evaluation Protocol	32
CHAPTER 5: RESULTS AND DISCUSSION	33
5.1 Summary of results and discussion	33
5.1.1 Retrieval evidence coverage	34
5.1.2 Historical factual-answer baseline	35
5.1.3 Evidence delivery and context audit	36
5.1.4 Production context repair and replay	37
5.1.5 Long-document summarization	38
5.1.6 Functional and runtime observations	39
5.1.7 Validity threats and error analysis	40
CHAPTER 6: CONCLUSION AND FUTURE SCOPE	41
6.1 Conclusion	41
6.2 Future scope	42
REFERENCES (APA format)	43–46
APPENDICES	47
A Reproducibility record	47
B Demonstration acceptance cases	48
C Academic integrity and required reports	49
D Submission and evidence checklist	50

LIST OF TABLES

Table and title	Page
Table 1.1 Objective and verification mapping	4
Table 3.1. Principal software components	13
Table 3.2. Evaluated laptop configuration	14
Table 3.3. Main work packages	15
Table 3.4. Cost categories and estimation basis	19
Table 5.1 Retrieval on 96 synthetic queries	34
Table 5.2 Historical generation outcomes before repair	35
Table 5.3 Historical baseline and production repair outcomes	37
Table 5.4 Requested facts retained in final summaries	38
Table 5.5 Historical resource and runtime context	39
Table A.1 Reproduction settings	47
Table B.1 Demonstration checks and recovery	48
Table C.1 Mandatory attachment register	49
Table D.1 Project evidence and preservation	50

LIST OF FIGURES

Figure and title	Page
Figure 4.1 Local processing and MCP architecture	20
Figure 4.2 Library view in the isolated local rehearsal	21
Figure 4.3 Persistent source, memory and conversation entities	22
Figure 5.1 Historical evidence loss and earlier research diagnostic	36

LIST OF ABBREVIATIONS

Term	Meaning
AFVM	All-fact value match
API	Application programming interface
BM25	Best Matching 25 ranking family
CES@k	Complete evidence success among top k results
CI	Continuous integration / confidence interval (context dependent)
FTS5	SQLite full-text search extension
FVM	Fact-value match
LLM	Large language model
MCP	Model Context Protocol
MRR	Mean reciprocal rank
OCR	Optical character recognition
RAG	Retrieval-augmented generation
RRF	Reciprocal rank fusion
RSS	Resident set size
VRAM	Video random-access memory

Chapter 1
Introduction

1.1 Introduction

LocalMind is a local-first knowledge copilot for files, project notes and deliberately saved memories. It combines a browser interface, a local language model, document retrieval and a custom Model Context Protocol (MCP) server. The intended user is a student or knowledge worker whose useful context is distributed across folders rather than one searchable project workspace.

Users connect selected folders, inspect extracted text, ask questions and save facts they want to retain. Originals remain at their existing paths; the application stores its index and conversational state locally. A new conversation can use persistent project memory without depending on unlimited chat history. These operations remain subject to approved-source, format and resource limits.

This updated report distinguishes three engineering stages: the historical evidence-delivery failure, a research-only diagnostic and the subsequently implemented production repair. Exact serialized-cost fitting is now applied to both local platform copies. A replay improves factual completion, but reused questions, passage-prefix losses and limited tool use prevent a general accuracy claim. The contribution remains inspectable local integration and reproducible evaluation rather than a new retrieval algorithm.

1.2 Problem statement

A project decision may be recorded in a Markdown note, an implementation detail in source code and a schedule in a PDF. Filename search helps when the user remembers suitable words. A conversational assistant without access to those records may produce a plausible answer from model parameters or require repeated uploads. Persistent local context addresses this practical fragmentation.

The engineering problem includes both obtaining evidence and delivering it intact to the model. Historical LocalMind retrieval found the expected material, but approximate clipping discarded the evidence message after JSON serialization expanded it. The production repair addresses that failure through exact serialized UTF-8 accounting. Remaining challenges include selecting facts beyond short passage prefixes and verifying the relationships asserted in generated answers.

The research question is how effectively a compact local model can use approved evidence through a custom MCP service on a constrained laptop. Retrieval, context retention, alias-based factual completion, summaries and resource demand are evaluated separately. The new production replay reuses frozen questions and is not held-out confirmation. Privacy means local processing after installation; zero recurring inference-API charges do not eliminate hardware, electricity, storage or maintenance costs.

1.3 Scope of research

The system covers approved folders containing supported text, Markdown, code, DOCX text and PDFs with extractable text. It provides indexing, hybrid search, extraction viewing, source-linked chat, summaries, explicit memories, project workspaces and saved plans. The Windows edition supplies platform-specific setup and filesystem safeguards. Both local copies now contain the same repaired inference code; the repair is uncommitted and is identified by its source hash.

Local drive means files already available on the laptop HDD, SSD or an explicitly approved accessible path. The application does not silently scan every partition. Arbitrary binary understanding, image-only PDF OCR, continuous watching, encrypted storage and autonomous code execution remain outside scope. Exact message fitting preserves bounded evidence input; it does not guarantee inclusion of every fact or correct generation.

1.4 Research hypothesis

H1 predicts complete expected evidence within five results on the synthetic corpus; this holds for lexical and hybrid retrieval. H2 predicts reliable complete factual answers; the repair improves completion to 7/24 and 3/24, leaving H2 unsupported as a reliability claim. H3 predicts an adaptive MCP-agent benefit over single-pass RAG; no additional model-selected tool executes, and the replay provides no such benefit. These findings concern reused synthetic questions.

1.5 Objectives

The primary objective is an inspectable local workflow: approve a folder, retrieve content, verify a source, retain a selected fact and reuse project context. A second objective is genuine MCP communication between the agent and persistent knowledge operations. A third is to preserve evidence across context fitting and evaluate failures with reproducible inputs.

The original experiment and its research diagnostic remain historical records. The repaired production code is verified separately through regression tests and a replay using the same question pairs. Passing these checks establishes the exercised mechanisms and bounded input retention. It does not prove complete answers, independent generalization or successful autonomous tool selection.

Table 1.1 Objective and verification mapping

Objective	Verification evidence
Local document access	Approved-root and extraction functional tests
Persistent memory	SQLite storage and project-scope tests
Protocol-based tools	Official SDK stdio MCP round trips
Retrieval quality	96 queries; evidence coverage at k
Answer reliability	Historical baseline and production replay; 24 positives + 8 unknowns
Platform support	Current Linux checks; historical native Windows CI

The acceptance evidence is intentionally mixed. Passing integration tests establishes that operations execute as specified; it does not prove that generated prose is correct. The report therefore preserves both the functioning infrastructure and the failed factual-answer experiment, and uses the latter to prioritize future engineering work.

1.6 Organization of the report

Chapter 2 reviews retrieval-augmented generation, sparse and dense search, tool use, memory and local-first software. It distinguishes established methods from integration choices and explains why retrieval, evidence delivery and final factual quality require separate evaluation.

Chapter 3 specifies software, hardware, functional requirements, work packages and costs. Chapter 4 follows folder approval, extraction, ranking, MCP communication, memory, model context and summarization. Its context-fitting description includes the applied production repair and the limitations that remain.

Chapter 5 separates the historical baseline, earlier diagnostic and new production replay. It reports unchanged retrieval measurements, repaired input retention, factual completion, historical summaries and platform validation. Chapter 6 records the completed repair and prioritizes remaining work. APA references and appendices preserve provenance, demonstration instructions and the status of required institutional attachments.

The report retains 60 physical pages, including cover and preliminary material. Front matter uses Roman numbering; the technical body restarts at page 1. The companion IEEE paper and raw records provide additional detail. Historical commits and results remain identifiable, while the current local repair is recorded separately by SHA-256 rather than attributed to an older Git commit.

Chapter 2

Literature Review

Retrieval-Augmented Generation and Dense Representations

2.1 Background

Retrieval-augmented generation connects a language model to evidence outside its parameters. Lewis et al. (2020) establish this combination as a way to use external knowledge during generation. For LocalMind, the practical implication is that current project files can supply context without retraining the chat model. The original research includes trained components and public tasks; its reported performance cannot be transferred to this different local pipeline.

Karpukhin et al. (2020) investigate dense passage retrieval through learned question and passage encoders. Reimers and Gurevych (2019) demonstrate efficient independent sentence representations for similarity comparison. These works explain the motivation for semantic retrieval when wording differs between a question and a document. LocalMind uses EmbeddingGemma instead of those encoders, so the cited studies provide foundations rather than an implementation identity or an expected accuracy figure.

The distinction matters for personal knowledge. Project names, identifiers, filenames, and technical terms may remain unchanged even when a question is paraphrased. Dense similarity should therefore be compared with exact lexical matching on the same extracted passages. Retrieval also precedes prompt construction: a correct search result is useful only if its evidence survives formatting and context limits. LocalMind consequently evaluates source coverage separately from final answer-value matching, preserving the possibility that retrieval succeeds while downstream generation fails. This staged view avoids treating a source card as proof of a grounded answer.

Lexical Retrieval and Reciprocal Rank Fusion

BM25 remains a useful lexical foundation because it combines term-frequency saturation with document-length normalization (Robertson & Zaragoza, 2009). Exact terminology is common in notes, specifications, and code. LocalMind implements lexical search with SQLite FTS5 across chunk text, title, and path. Its field weights are 1.0, 2.0, and 0.3. SQLite negates the conventional BM25 expression, so better matches have numerically lower scores (SQLite developers, n.d.).

Reciprocal rank fusion combines positions from separate rankings rather than calibrating their raw score scales (Cormack et al., 2009). LocalMind adds equal contributions of 1/(60 + rank) from lexical and dense lists, with absent candidates contributing zero. This is an established fusion rule, not a new algorithm. The dense component uses positive cosine similarity over compatible local embeddings. If embeddings are unavailable, lexical search remains usable without hosted services.

The combination is plausible but must be tested against both components. A dense result can improve recall while lowering the position of an exact lexical match; fusion therefore need not dominate every query. On the small identifier-rich LocalMind corpus, lexical and hybrid methods both reach complete-evidence success for 96 of 96 questions within five results. Hybrid adds no gain at that cutoff. These conditional findings support retaining a strong baseline rather than presuming that a more complex retrieval method is universally better. They do not establish answer accuracy or large-archive performance.

Tool-Using Agents and Bounded Actions

A conversational assistant becomes more useful when it can obtain missing information through defined operations. ReAct studies interaction between reasoning and actions (Yao et al., 2023), while Toolformer investigates learning when and how to call APIs (Schick et al., 2023). LocalMind draws on this general tool-use direction. It uses an existing pretrained model and runtime checks; it does not reproduce Toolformer's training method or experimentally expose internal reasoning.

The implemented agent receives six read schemas for search, document reading, recall, document listing, project listing, and conversation lookup. Its loop permits at most four model/tool rounds and six additional tool executions after initial retrieval and recall. A final synthesis call can run with tools disabled. The host rejects requests outside its allowlist. These bounds make execution finite and keep generated requests from becoming arbitrary shell or filesystem operations.

An available tool is not evidence that the model used it effectively. Neither the historical runs nor the repaired production replay execute additional model-selected tools. The current agent therefore shows bounded protocol capability without an observed adaptive retrieval advantage. Tool schemas also consume prompt space. Exact-cost fitting now prevents the demonstrated avoidable evidence drop, but it cannot make short passage prefixes contain missing facts. Future experiments need tasks requiring further reads and must inspect whether those reads deliver the requested values. Successful protocol exchange, retained input and factual output remain distinct properties.

Persistent Memory Architectures

Persistent memory is broader than saving a chat transcript. MemGPT explores hierarchical context and virtual memory management (Packer et al., 2023). MemoryBank considers user-oriented memory and updating or forgetting information (Zhong et al., 2024). LongMem augments language modeling through a learned memory architecture (Wang et al., 2023). These designs illustrate different mechanisms; their benefits cannot be assumed for a system that simply stores relational records.

LocalMind keeps explicit notes, project descriptions, sessions, and conversation messages in SQLite. Notes can have a kind and an optional project association. Memory lookup uses bounded substring filtering, while conversation search uses FTS5. The implementation has no knowledge graph, learned consolidation, automatic preference extraction, or forgetting schedule. A record surviving a restart establishes persistence, but does not guarantee that it will be retrieved or fit into a later prompt. This separation is central to an accurate account of personalization.

Related agent research adds further distinctions. Generative Agents includes memory and reflection for simulated behavior (Park et al., 2023), while Reflexion retains textual feedback across trials (Shinn et al., 2023). LocalMind does not implement those reflection loops. Its user explicitly controls memory writes, and retrieved text cannot directly invoke the current-user memory command path. This restriction keeps stored knowledge inspectable and avoids presenting ordinary conversation history as autonomous learning. Future evaluation should measure whether saved context improves later assistance, including stale preferences and conflicting notes, rather than equating database presence with useful long-term memory.

Local Ownership, MCP, and Untrusted Documents

Local-first software emphasizes ownership, offline access, and user agency, alongside a broader vision of collaborative replication (Kleppmann et al., 2019). LocalMind implements single-laptop processing rather than replicated collaboration. Its application, database, approved folders, and model server can operate locally after installation. Model and dependency downloads need Internet access initially. Avoiding paid inference APIs therefore removes an API fee, not hardware, storage, electricity, installation bandwidth, or maintenance costs.

MCP supplies an interoperable host, client, and server interface for tools and resources (Model Context Protocol contributors, 2025). LocalMind negotiates an actual SDK session with a separate stdio server and exchanges structured results. The server exposes both reads and explicit mutations, but the built-in model receives only read tools. Protocol annotations describe behavior; they do not enforce authorization. External clients need their own controls and can transmit returned content outside the local boundary.

Privacy and security must remain separate claims. Indirect prompt injection can embed instructions in retrieved material (Greshake et al., 2023). Local inference does not make such content trustworthy. LocalMind marks it as reference data and limits model capabilities, yet reports no adversarial security proof. The database is unencrypted, and software with the user's permissions can read it. Folder approval also bounds access: the application does not silently scan the entire drive. These choices support a controlled workflow without establishing confidentiality against compromised local software or every possible document instruction.

Evaluation: Evidence, Context, and Citations

BEIR emphasizes heterogeneous retrieval tasks and the continuing value of lexical baselines (Thakur et al., 2021). MTEB evaluates embeddings across varied tasks and languages (Muennighoff et al., 2023). These references motivate broader future testing; their scores are not LocalMind results. A small English corpus with recurring project identifiers cannot establish general personal-drive performance. Evaluation needs an explicit corpus, frozen questions, correct source paths, and clearly defined success criteria.

RAGAs separates retrieval, faithfulness, and answer quality (Es et al., 2024). Citation research further asks whether a reference supports a generated claim (Gao et al., 2023). LocalMind consequently distinguishes complete-evidence success, fact-value matching, source-index validity, and gold-path coverage. A correct citation index can accompany a wrong date or incomplete answer. Its deterministic alias scorer also has limits: a negated or misattributed value can match, and a valid unlisted paraphrase can fail. No language model judges the reported quantitative results.

Lost in the Middle shows that long context does not guarantee uniform use of relevant information (Liu et al., 2024). The historical LocalMind summary experiment likewise separates section visitation from final value retention: fifteen sections were processed, but only ten of twenty-four requested values survived. This does not reproduce the earlier study’s mechanism. The production context repair replay did not repeat those summary tasks. Retrieval, prompt delivery and output claims therefore require separate records; a completed serialization repair should not be reported as a demonstrated solution to long-document summarization.

Research Gap and LocalMind's Positioning

2.2 Summary of literature review and research gap

Prior work establishes retrieval-augmented generation, sparse and dense matching, fusion, tool use, memory and local ownership (Lewis et al., 2020; Cormack et al., 2009; Yao et al., 2023; Kleppmann et al., 2019). LocalMind addresses practical integration on a constrained laptop, not an absence of earlier memory assistants. It combines approved-folder access, persistent context and an inspectable MCP interface.

The engineering contribution now includes an applied exact serialized UTF-8 context repair alongside local serving, SQLite retrieval and complete-text pagination. The repair adds no learned retrieval or memory algorithm. Persistence still differs from useful recall, and source references still differ from supported claims (Packer et al., 2023; Es et al., 2024; Gao et al., 2023).

The unchanged retrieval experiment covers all 96 questions within five lexical or hybrid results. Historically both generators completed 0/24 positives after evidence loss. The repaired production replay retains references for all 64 requests and reaches 7/24 single-pass and 3/24 agent AFVM, matching earlier diagnostic totals. Reused questions are not independent confirmation, and no additional model-selected tool executes. Prefix selection, summary retention, heterogeneous archives and semantic assessment remain open work. This positions LocalMind as a repaired, auditable prototype with measured limitations.

Chapter 3

Software Requirements Specification

3.1 Software Tool Platform/ Tools/Framework used

The software requirements translate the need for private, persistent assistance into components that can run on a personal laptop. LocalMind combines a browser interface, a local HTTP application, document processing, durable storage and an independent MCP memory server. The browser presents approved folders, conversations, memories and projects. The application coordinates these functions without requiring a paid inference service.

Python provides the common implementation language. FastAPI and Uvicorn serve the local interface, while the official MCP SDK handles protocol initialization, tool discovery and tool execution. SQLite stores both structured records and searchable text. NumPy supports vector calculations, and pypdf extracts textual PDF content. DOCX processing uses Python ZIP and XML libraries. Ollama supplies installed local language and embedding models.

Table 3.1. Principal software components

Component	Purpose
FastAPI / Uvicorn	Local browser application
SQLite / FTS5	Durable records and lexical search
MCP SDK	Separate stdio memory service
Ollama / NumPy	Local inference and vector scoring
pypdf / ZIP / XML	Supported document extraction

Hardware and Execution Environment

3.2 Hardware tools

The target configuration is an existing laptop with an AMD Ryzen 7 250 processor, an NVIDIA RTX 5050 Laptop GPU and nominal 16 GB system memory. This configuration motivates a compact quantized language model and a local embedding model. It also motivates explicit limits on document size, extraction, context length and tool execution. Available memory must support the desktop, model runtime, application and temporary processing together.

The measured Linux environment exposed approximately 14,768.9 MiB system memory and 8,151 MiB GPU memory. These operating system observations differ from marketed capacity because of reporting units and reservations. The evaluated chat model was Qwen3 4B in Q4_K_M format; EmbeddingGemma generated 768-dimensional document vectors. Model dimensions are metadata of this deployment rather than fixed assumptions in the storage layer.

Table 3.2. Evaluated laptop configuration

Item	Observed configuration
Processor	Ryzen 7 250; 8 cores, 16 threads
Graphics	RTX 5050 Laptop; 8,151 MiB VRAM
System memory	Nominal 16 GB; 14,768.9 MiB visible
Model runtime	Ollama 0.35.0
Operating system	Linux 7.0.0-38-generic

3.3 Work Breakdown Structure

The work breakdown begins with scope definition: approved local files, supported document types, persistent memory and a browser interface suitable for a nontechnical user. Architectural work then establishes the boundaries between the API, indexing library, SQLite store, Ollama adapter and MCP process. Defining these boundaries early allows persistence and protocol behavior to be checked independently of model output.

Implementation proceeds through source validation, bounded readers, parsers and chunking before retrieval and generation. Storage work supplies transactional updates, lexical indexes and project relationships. The MCP server exposes these capabilities through explicit tools. The agent then consumes those tools, while the interface provides source approval, progress information, conversation history and memory management. Platform launchers are a separate deployment task.

Table 3.3. Main work packages

Package	Concrete output
Requirements and design	Scope, component boundaries, schema
Knowledge processing	Safe extraction and incremental index
Memory and agent	MCP tools and bounded execution
Application and deployment	Browser interface and local launchers
Validation and reporting	Tests, benchmark records and analysis

Functional Requirements: Sources and Retrieval

3.4 Functional Requirements

The first functional requirement is explicit source approval. The user selects an existing folder on a local disk, and the application records that scope before indexing. The scanner must reject protected or overlapping roots and avoid symbolic links, excluded directories and known secret filenames. Removing an approved source must remove its associated indexed documents through the stored ownership relationship.

The second requirement is useful extraction from supported files. Text, Markdown and code are decoded as text; PDFs and DOCX files are processed through their supported extraction paths. Successful extraction produces a complete stored text snapshot and bounded passages with exact character offsets. A rescan must update changed files, preserve valid unchanged work and remove absent documents only after a complete traversal.

The third requirement is retrieval with inspectable provenance. A query may combine lexical and semantic candidates, but every returned passage must identify its document and location metadata. Before delivery, the application checks that the source still belongs to an approved root and matches its indexed size and modification time. Document browsing and paginated extraction reads allow the user to inspect evidence beyond a short answer.

Functional Requirements: Memory and Assistance

3.4 Functional Requirements — continued

Persistent assistance requires more than document search. Users must be able to create, edit and delete explicit memories, optionally associate them with projects and retrieve them in later conversations. A memory edit retains its identifier and original creation time. Sessions and messages persist conversation content, citations and execution traces, allowing relevant user and assistant messages to be searched across saved chats.

The assistant must retrieve local evidence before ordinary generated answers and provide an extractive mode when a usable chat model is unavailable. It may request additional read-only MCP tools within configured limits. Whole-document summarization must follow all stored extraction pages rather than substitute a few search hits. Project assistance must create an editable plan associated with a project, while keeping generated output inside the application data directory.

These requirements have practical boundaries. Projects organize memories and conversations; they do not create separate filesystem permissions for each project. Planning writes one PLAN.md file and does not generate, compile or execute a software repository. Similarly, the implemented tool loop provides capability for additional retrieval, but the fresh benchmark observed no additional model-selected tool calls. Functional availability and demonstrated model behavior are reported separately.

3.5 Non Functional Requirements

Privacy is a deployment property supported by explicit technical boundaries. In the normal local interface, document extraction, embeddings, inference and persistence remain on the laptop. Ollama endpoints are limited to HTTP loopback addresses, redirects are disabled and proxy environment variables are ignored by the adapter. Downloading dependencies or models is an initial network activity and must be distinguished from later document processing.

Reliability requires durable transactions and visible failure behavior. SQLite foreign keys preserve ownership relationships, while write-ahead logging supports the API and MCP processes sharing one database. Failed document replacement rolls back instead of leaving partial chunks. Resource limits bound file reads, extracted text, scan candidates, passage size and agent execution. Embedding failure leaves lexical indexing available; unsupported or unreadable documents produce explicit errors.

Usability requires understandable progress, source citations and editable memory rather than hidden state. Maintainability requires clear modules and repeatable tests. Security remains limited to a trusted local-user setting: the prototype has no application authentication, encrypted database or operating-system sandbox. Other local processes can access the same files or loopback service. These residual risks must be documented without treating local execution alone as complete security.

3.6 Project Cost Estimation

LocalMind avoids recurring charges to a hosted inference API by using installed open-source components and downloaded local models. The proposed deployment reuses the student laptop and its existing storage. Under that assumption, the additional hardware purchase cost can be zero. This statement concerns incremental project expenditure; it does not imply that computing hardware, electricity, internet access or development effort are free.

A transparent estimate separates fixed resources from variable operating costs. Storage must accommodate model weights, extracted document text, passage records, full-text indexes and vectors. Electricity cost can be calculated from measured average power, operating hours and the local tariff. Network expenditure depends on the initial downloads and future updates. No power measurement or market-price survey was conducted in the reported benchmark, so numerical estimates for these items would be unsupported.

Table 3.4. Cost categories and estimation basis

Category	Estimation basis
Inference API	No paid hosted API required
Existing laptop	No additional purchase assumed
Electricity	kW × operating hours × local tariff
Storage / downloads	Actual capacity and connection usage
Maintenance	Student or user time; not measured

Chapter 4

Methodology

4.1 System Architecture and Data Model

The methodology separates deterministic knowledge handling from probabilistic language generation. Approved documents enter a controlled extraction and indexing pipeline. SQLite stores their text, metadata and vectors. A separate MCP server exposes retrieval and memory operations, while the agent uses those operations to assemble evidence for a local model. The browser interface provides the human point of control over approved sources and explicit memory.

Browser requests use loopback HTTP; agent knowledge operations use a separate MCP stdio process. Chat and embedding requests reach the locally installed Ollama runtime. Source originals remain in approved folders.

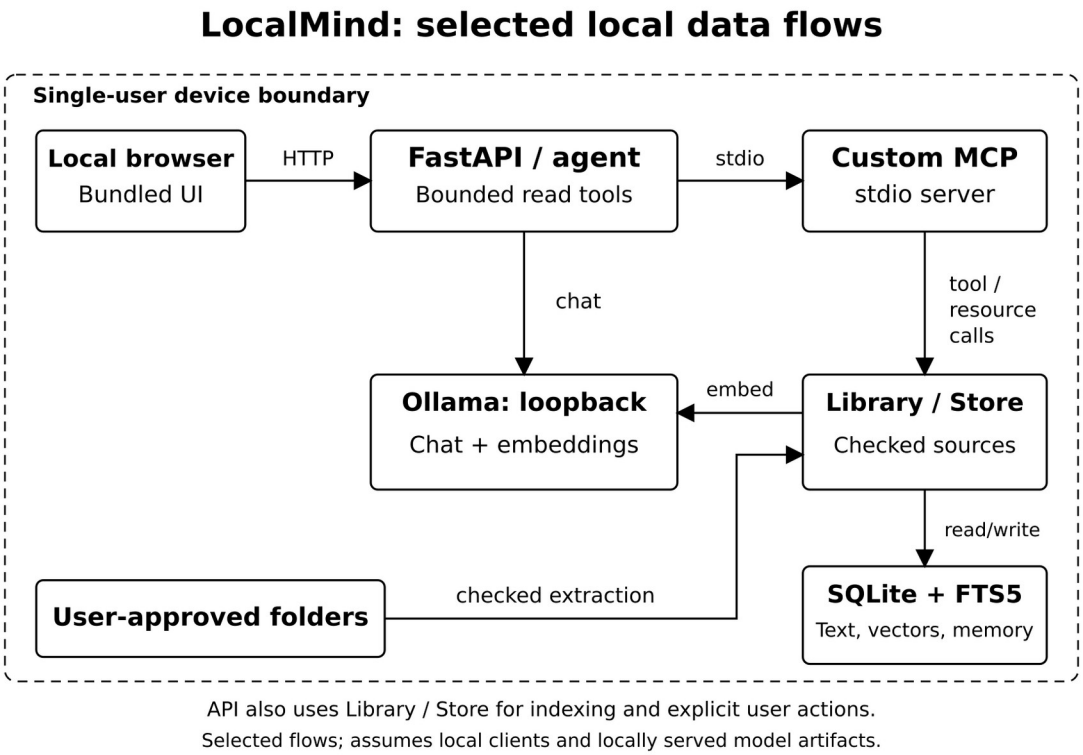


Figure 4.1 Local processing and MCP architecture

4.1.1 User interface and data flow

The interface has five main areas: Copilot, Library, Memory, Projects and Settings. Library connects an explicitly chosen folder and opens extracted documents. Memory stores selected facts. Projects organize a workspace and its persistent notes; Settings confirms installed local models. Copilot combines a conversation with retrieved evidence and an inspectable tool trace.

The screenshot shows the isolated Lighthouse rehearsal, using only the two supplied example files. It is interface evidence rather than a replacement for the controlled accuracy experiment. A source card can be opened to verify a claim against the reader. The demonstration guide gives exact navigation and recovery steps.

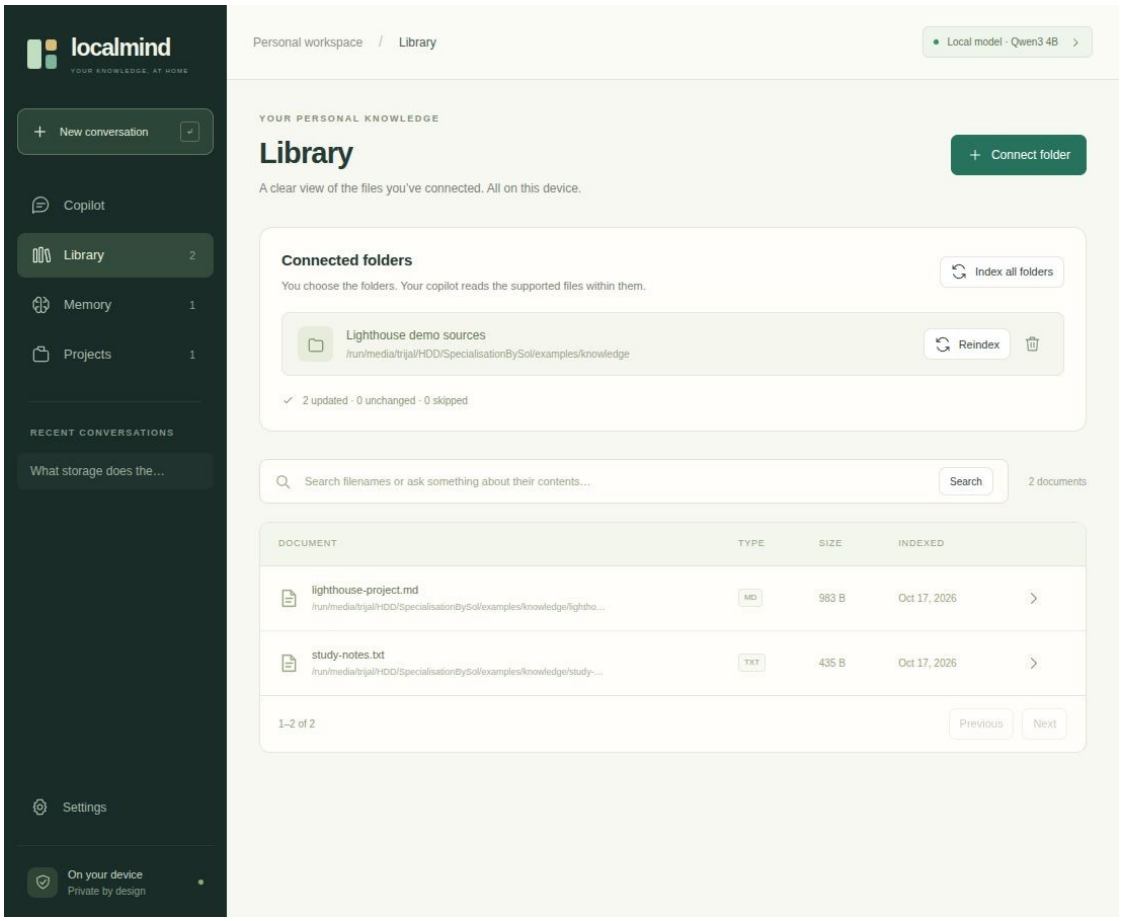


Figure 4.2 Library view in the isolated local rehearsal

Persistent Data Model

4.1 System Architecture and Data Model — continued

The database distinguishes source knowledge from personal working context. A root records an approved folder. Each document belongs to one root and stores its path, extracted text, extension, size, modification timestamp and indexing time. Each chunk belongs to a document and stores an exact text interval, an optional PDF page and an optional embedding with model and dimension metadata.

Memories and sessions may belong to a project; messages belong to sessions. Citation and trace JSON remain attached to messages. These are relational records, not an inferred knowledge graph.

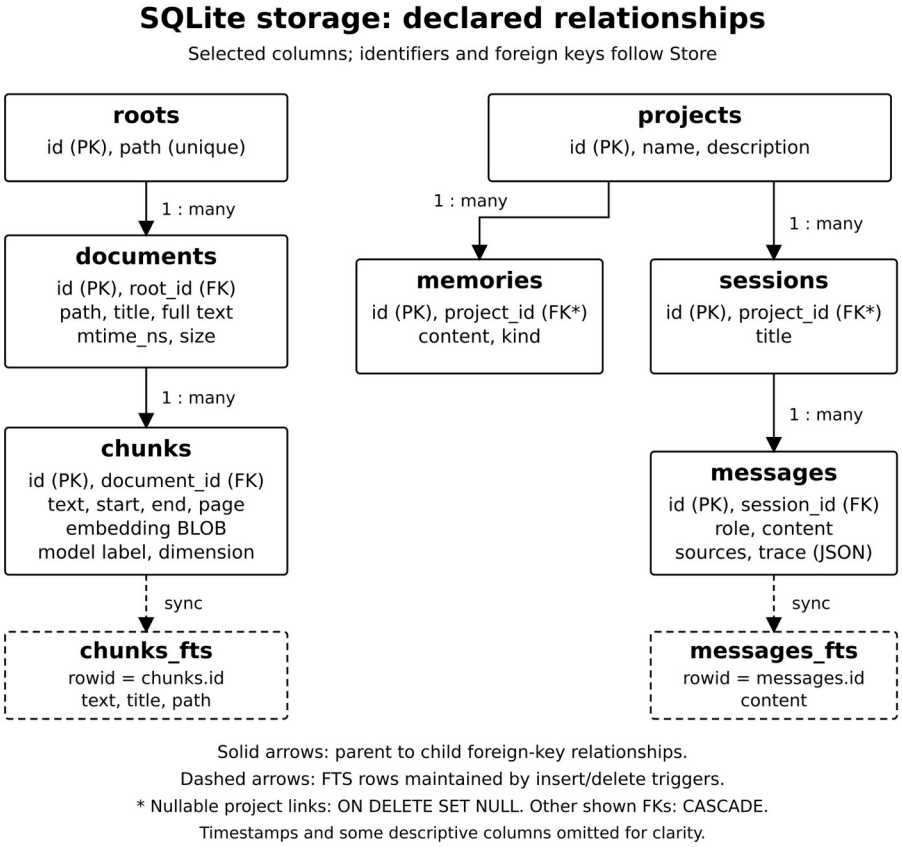


Figure 4.3 Persistent source, memory and conversation entities

4.1 System Architecture and Data Model — continued

Each store operation opens a short-lived SQLite connection with foreign keys enabled and a thirty-second busy timeout. Write-ahead logging allows the API and MCP processes to share a database without introducing an additional database service. Writes remain transactional: replacing a document deletes its old chunks and inserts the new extraction and passages in one operation. An exception rolls back the replacement.

The full-text indexes use FTS5 with the unicode61 tokenizer. The chunk index contains text, document title and path; insert and delete triggers keep it aligned with chunk rows. Conversation search has a separate content index and includes user and assistant messages. Explicit memory search instead uses a bounded substring query over recent notes. A single phrase such as persistent memory therefore covers several deliberate mechanisms rather than one universal vector index.

Embeddings are stored as little-endian float32 binary values with their model label and dimension. Inserts reject nonfinite values and inconsistent dimensions within a document. Search compares only compatible vectors. This prevents an embedding-model change from silently mixing unrelated vector spaces. It does not eliminate the need to rescan or backfill embeddings when the configured model changes, nor does it establish semantic quality by itself.

Approving Sources and Restricting Scope

4.2 Local Ingestion and Hybrid Retrieval

Ingestion starts with an explicit folder choice. The library expands and canonicalizes the path, verifies that it is an existing directory and rejects prohibited locations. Overlapping approved roots are refused to avoid ambiguous ownership and duplicate indexing. The application data directory is excluded, preventing its own database, plans and runtime state from being ingested as ordinary source material.

The scanner does not follow symbolic links. It also omits hidden entries, dependency and build folders, system-related directories and known secret filenames. These exclusions reduce accidental collection but do not inspect the meaning of every document. A password written inside an otherwise allowed Markdown file remains possible. Source approval therefore grants access to eligible content inside that folder and should be treated as a meaningful user decision.

Platform handling follows the same scope goal through different filesystem APIs. Linux uses bounded descriptor-based reads with link avoidance. The Windows edition adds native checks for junctions, reparse points, network paths and handle destinations, while allowing ordinary folders on local fixed or removable drives. The approved scope is a document folder, not an unrestricted permission to read every system file on the laptop.

4.2 Local Ingestion and Hybrid Retrieval — continued

A supported file is read within a configured twenty-mebibyte limit. The reader accepts regular files and compares file size and modification time before and after reading. If the source changes during ingestion, the operation fails and asks for a later rescan. The scanner also limits supported-file candidates to twenty thousand, while extraction limits each document to five million characters and PDFs to two thousand pages.

Text decoding recognizes Unicode byte-order marks, tries UTF-8 and falls back to CP1252 where appropriate. Binary-content checks reject common nontext inputs, and line endings are normalized. PDF extraction uses pypdf and records one-based page boundaries. Scanned images without extractable text need a separate local OCR step; the application does not claim to understand them automatically.

DOCX extraction reads supported XML parts from the ZIP package, including paragraphs, table cells and available headers, footers, footnotes and endnotes. It preserves textual tabs and breaks without reproducing the original page layout or embedded graphics. The stored snapshot is therefore the complete successful extraction, not a visual or byte-identical copy. This distinction is necessary when describing full-document access and summarization.

Page-aware Chunking and Embeddings

4.2 Local Ingestion and Hybrid Retrieval — continued

After extraction, the library divides text into passages while retaining exact start and end character offsets. The normal passage limit is 1,800 characters with 200 characters of overlap. When a passage would end before the document boundary, the algorithm prefers a newline or sentence delimiter in the latter half of the candidate interval. This reduces abrupt splits without requiring an additional language-processing model.

PDF page intervals are processed separately, so one chunk never crosses two PDF pages. Empty passages are omitted, and a progress guard prevents overlap from causing an infinite loop. Character counts are intentionally described as characters rather than tokens because the local model tokenizer can produce different lengths for different content. Stored offsets allow passage text to be checked against the document snapshot exactly.

The local embedder processes batches of sixteen chunk texts. A document receives vectors only when all its embedding batches succeed with compatible finite dimensions. The first embedding failure disables further embedding attempts during that scan, allowing lexical indexing to continue. On a later rescan, unchanged documents can have missing or model-mismatched embeddings backfilled without reparsing their source. This fallback preserves access while making semantic retrieval availability explicit.

4.2 Local Ingestion and Hybrid Retrieval — continued

The query is trimmed to 1,024 characters and converted into at most twenty-four distinct Unicode alphanumeric tokens. Quoted tokens are joined with OR for FTS5. Lexical ordering uses native BM25 over passage text, title and path, with respective field weights of 1.0, 2.0 and 0.3. This stage is useful for exact project names, identifiers and terms found in local documents.

For semantic retrieval, the store scans compatible vectors in batches of 128 rows and computes cosine similarity. Invalid vectors, zero norms and nonpositive similarities are excluded. A bounded heap retains the strongest candidates. The implementation performs exact scanning rather than approximate nearest-neighbor search, so semantic work grows with the number of compatible vectors. No measured large-corpus scalability claim follows from this design.

Hybrid ordering applies equal-weight reciprocal-rank fusion: a candidate contributes 1/(60 + rank) from each list in which it appears. Ranks start at one; an absent list contributes zero. Ties use chunk identifiers. The ranking depth is min(160, max(40, 4L)), where L is the requested store limit. Fusion scores are ordering values rather than probabilities. The library subsequently filters stale or disallowed sources before returning the requested number of passages.

Incremental Indexing and Source Freshness

4.2 Local Ingestion and Hybrid Retrieval — continued

A rescan compares each supported file with its stored size and nanosecond modification timestamp. Matching metadata allows the existing extraction to be reused, except when embeddings require backfilling. Changed documents are parsed and replaced transactionally. This avoids repeatedly processing every unchanged file while keeping source ownership, passage text and full-text entries consistent after a successful update.

Deletion handling depends on completing the folder traversal. The scanner collects seen paths and removes absent document rows only when enumeration finishes without traversal errors or the configured candidate limit. Otherwise pruning is suppressed. A failed parse can leave an older snapshot in storage, but retrieval independently checks the live source signature and excludes it when metadata differs. The MCP document reader applies corresponding approval and freshness checks.

Freshness is deliberately limited to metadata validation. A content change that preserves both size and modification time can evade detection because the index does not use a cryptographic content hash. There is also no filesystem watcher; the user initiates rescans. These limits distinguish immediate rejection of a visibly stale snapshot from a guarantee that every stored byte always matches a changing disk.

MCP Server and Client Execution

4.3 MCP Memory and Agent Execution

The custom memory server uses the official SDK FastMCP implementation with stdio transport. The application client starts a separate Python process, establishes a ClientSession, initializes the protocol, discovers tools and issues tool calls. Both processes use the configured SQLite database. This provides actual MCP interoperability without opening a network listener for the memory service.

Ten tools cover knowledge search, document listing and reading, explicit memory creation and recall, forgetting, project listing and creation, conversation search and approved-source listing. Two resources expose an overview and project information. Document reading takes an indexed identifier and a bounded offset and limit; it returns total length and a next offset. The default read is 12,000 characters and the maximum is 50,000 per call.

The built-in agent receives only six read-only tool schemas. External MCP clients may access additional write tools, while the browser API can write memories directly. Tool annotations communicate intended behavior but do not act as authentication. Accordingly, the MCP boundary is a structured capability interface for a trusted local client. It is not an encrypted storage service or a permission system that makes arbitrary external clients trustworthy.

4.3 MCP Memory and Agent Execution — continued

For ordinary questions, the agent loads at most ten recent messages, then makes initial knowledge_search and recall calls. It retrieves five passages and selected project or global memories. Each initial passage is still cropped to its first thousand characters. The production context repair does not redesign this evidence-selection stage.

The two initial calls are outside the six-additional-tool budget. Ordinary turns permit at most eight actual MCP tool executions and five model calls: four planning calls plus one final synthesis. Summarization is a separate branch. Active-project recall and conversation search are scoped to that project; file retrieval remains across globally approved roots. Only the current user message can trigger the explicit remember command.

The repaired fitter uses budget $B = C - \min(1536, \text{floor}(C/3)) - 256$, where C is context size. It accounts for serialized UTF-8 messages, metadata, tool schemas and shortening markers, pins the first system and latest user messages and finds the longest fitting complete-character prefix. Small remaining budgets are checked exactly. Tool-call metadata remains whole. This code is applied locally; the separate production replay retains initial references for all 64 requests within the configured bound.

Whole-document Summaries and Project Plans

4.3 MCP Memory and Agent Execution — continued

Whole-document mode reads every page of the stored extraction through MCP pagination. At the default context setting, the agent divides text into sections of approximately four thousand characters and generates partial summaries. It then combines those partial outputs through iterative merging. This path differs from ordinary retrieval, where only a limited set of relevant passages is selected.

Processing every section establishes visitation, not complete factual retention. Intermediate inputs can be cropped and generation may omit or misstate facts. The historical summary experiment processed fifteen sections with eleven model calls per task and retained ten of twenty-four target facts. Those summary tasks were not repeated in the production question-answer repair replay. Summaries therefore remain editable assistance that should be checked against complete stored extracts.

Project planning is similarly bounded. The agent retrieves three passages, combines them with relevant project and global memories and generates a Markdown plan. The saved output is projects/<id>/PLAN.md beneath the application data directory. Regenerating the plan replaces that file. The feature helps organize objectives and next steps, but it does not scaffold an executable project, modify arbitrary user folders or run generated commands.

4.4 Privacy Controls and Evaluation Protocol

The privacy boundary includes approved folders, the database, application, MCP process and local model runtime. Loopback validation, disabled redirects, proxy avoidance and local-model checks reduce accidental remote inference. Source policy bounds reads, but the application assumes a trusted user and provides no API authentication, encrypted database, operating-system sandbox or comprehensive prompt-injection protection.

Evaluation separates unchanged retrieval, historical generation, a research-only diagnostic and the current production repair replay. The corpus contains 39 Markdown documents and 141 chunks. Retrieval uses 96 answerable questions; generation reuses 24 selected positives and eight unknowns per method. The new replay verifies actual production code, reference retention and serialized-input bounds. Three historical summary tasks were not rerun for this repair.

For CES@k, each expected fact needs at least one accepted alias in a retrieved chunk from its exact labeled source path among the first k results. AFVM requires every requested value in generated text; macro FVM averages fact coverage per positive question. These aliases do not prove semantic entailment. Current regression checks and the 64-request replay verify the repair, while reused questions and historical native Windows CI limit broader conclusions.

Chapter 5

Results and Discussion

5.1 Summary of results and discussion

The frozen LM-Synth v1 corpus contains 39 Markdown files, 201,518 extracted characters and 141 chunks. Twelve fictional project families supply 36 ordinary documents; three dossiers support summary tasks. Retrieval evaluates 96 questions: 48 lexical, 36 paraphrase and 12 cross-document queries. Generation uses 24 selected positives and eight absent-value questions per method.

Three conditions must be distinguished. The original production snapshot lost evidence and completed no positive answers. A subsequent research-only exact-fit diagnostic tested the proposed mechanism. The current production repair is verified through a new 64-request replay without fitter substitution. The same corpus and question pairs are reused; improvement is development evidence, not held-out confirmation.

Scoring uses accepted aliases and exact expected source paths. Complete evidence success requires all requested facts in matching source chunks. All-fact value match requires every requested value in an answer; macro FVM averages matched fractions per positive question. Neither proves semantic entailment or penalizes every contradiction. The small synthetic project families also limit independence and generalization. Historical retrieval, summary and resource results are labeled separately from the production repair replay.

5.1.1 Retrieval evidence coverage

The unchanged frozen retrieval experiment measures Complete Evidence Success at five candidates (CES@5). Each expected fact needs an accepted alias in a chunk from its exact labeled source path. Lexical and hybrid retrieval meet this criterion on all 96 queries; dense retrieval meets it on 91. Its five misses occur among paraphrase questions. These figures describe retrieval evidence, not final answers.

Table 5.1 Retrieval on 96 synthetic queries

Retriever	CES@5	MRR@10
Lexical FTS5	96/96	0.9948
Dense cosine	91/96	0.8755
Hybrid RRF	96/96	0.9495

Hybrid retrieval does not outperform lexical retrieval here. MRR uses the first chunk containing any expected fact, not all cross-document facts. Median production ranking takes 4.02 ms lexical and 4.76 ms hybrid, excluding query embedding (13.60 ms median). Dense control rankings are computed in memory and are not comparable service-latency measurements.

The result supports an inexpensive inspectable search layer for this workload. It does not establish performance on OCR, multilingual notes, large code repositories, ambiguous names or personal archives. Those require separate datasets and relevance judgments.

5.1.2 Historical factual-answer baseline

Before the production repair, both single-pass RAG and the MCP agent matched no complete answers among 24 positive questions. Macro fact-value match was also zero. Each method returned the requested abstention phrase on all eight unknown questions and on every positive question. These are historical results of the original snapshot, not the current repaired application.

The requests completed without runtime exceptions, but their document-evidence messages did not survive context fitting. Source cards and successful MCP transport therefore coexisted with absent model input. No additional model-selected tool call executed beyond the agent's initial search and recall. The baseline is preserved in its original result directory so that the later diagnostic and production repair cannot silently replace an unsuccessful outcome.

Table 5.2 Historical generation outcomes before repair

Historical condition	Complete positives	Unknown abstentions
Original single-pass RAG	0/24	8/8
Original MCP agent	0/24	8/8

The 24 positives are clustered within synthetic project families and do not establish an arbitrary-drive population failure rate. The relevant engineering result is narrower: the original evidence-delivery implementation failed on this sample. The current replay reported next uses separately recorded repaired production code.

5.1.3 Evidence delivery and context audit

The historical audit found all 32 required fact slots in full returned passages for 24 positive questions. Keeping only the first 1,000 characters per passage reduced coverage to 15/32 slots and seven complete questions. Approximate context fitting then removed the entire evidence block, leaving zero slots. Figure 5.1 documents that earlier failure and research diagnostic.

The production repair now finds the longest complete-character prefix that fits the actual serialized UTF-8 message cost. Accounting includes JSON escaping, metadata, the shortening marker and tool schemas, while preserving pinned system and current-user messages. It removes fixed small-budget rejection heuristics and keeps tool-call metadata whole. All 64 production replay requests retain initial reference input and satisfy the configured serialized-input bound.

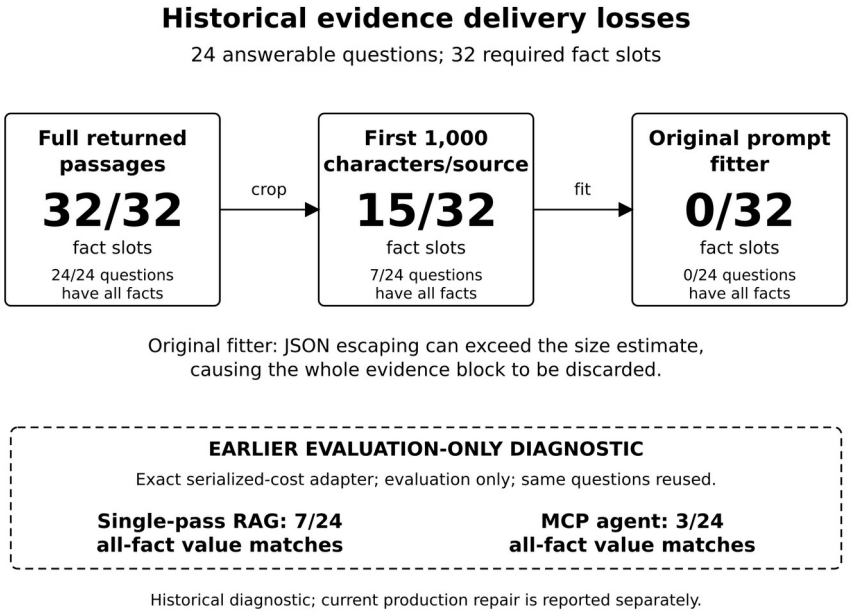


Figure 5.1 Historical evidence loss and earlier research diagnostic

5.1.4 Production context repair and replay

The new replay exercises actual repaired production code with real MCP calls and no diagnostic fitter substitution. Both methods answer the same 24 positives and eight unknowns used earlier. All 64 requests retain initial reference evidence, remain within the configured serialized-input bound and complete without request errors.

Table 5.3 Historical baseline and production repair outcomes

Condition	Complete positives	Macro FVM
Historical single-pass	0/24	0%
Repaired single-pass	7/24 (29.17%)	37.50%
Historical MCP agent	0/24	0%
Repaired MCP agent	3/24 (12.50%)	27.08%

Macro FVM averages fact coverage per positive question. The earlier research-only diagnostic also produced 7/24 and 3/24; those records remain historical and separate. Matching totals do not make this replay independent confirmation. The production repair is local and uncommitted, with both platform copies identified by the same source SHA-256.

The repaired single-pass method uses the abstention phrase on 8/8 unknowns; the agent does so on 7/8. Each response uses one model call, and neither method executes an additional model-selected tool. Limited prefix coverage and semantic mistakes remain; the results establish improved evidence delivery without demonstrating general reliability or an adaptive agent advantage.

5.1.5 Long-document summarization

The historical summary experiment used a route that reads sections through MCP, summarizes individual sections and merges intermediate notes. All 15 expected sections across three dossiers were processed. This verifies traversal coverage, but the final summaries retained only 10 of the 24 requested fact values. Reading every section is therefore a necessary process property rather than a guarantee of complete output.

Table 5.4 Requested facts retained in final summaries

Dossier	Sections read	Values retained
Morrow Annex	5/5	3/8
Oriole Basin	5/5	7/8
Tamarisk Span	5/5	0/8
Total	15/15	10/24 (41.67%)

Each historical summary required 11 model calls and approximately 56.75–64.38 seconds. Some final outputs claim that facts are absent even though the source contains them. Such statements require source verification and must not be repeated as established document properties.

The production repair replay evaluated question answering, not these summary tasks; the earlier 10/24 result remains historical. Structured field extraction and source-linked checks before narrative merging remain future work. A summary should be presented as an editable draft and verified against the reader, even when every source section was visited.

5.1.6 Functional and runtime observations

Current Linux validation passes 62 tests. The Windows checkout tested on Linux passes 99, with six native Windows checks skipped. These are not new native Windows results. Historical CI on Windows Server 2025 passed 86 tests per interpreter on Python 3.12.10 and 3.13.15 using fake Ollama; actual Windows 11 GPU inference remains unverified.

Nineteen new regression cases cover escaping, Unicode, exact boundaries, tiny budgets, input immutability, tool metadata and outgoing payloads. Fourteen of the initial fifteen cases failed before repair. Independent review also exercised 3,000 deterministic randomized budget cases. Both local platform copies contain identical repaired inference code and regression tests.

Table 5.5 Historical resource and runtime context

Historical observation	Measured value / qualification
Indexing	39 documents / 141 chunks; 12.875 seconds
Process RSS peak sum	8,258.89 MiB; shared pages may double count
Total GPU peak	5,317 MiB; desktop included
Resource sampling	239 samples in original experiment

The new replay median response times are 0.686 seconds for single-pass and 1.237 seconds for MCP. They concern reused questions and mixed successful or incomplete outputs, not a controlled speed comparison. Historical resource peaks above were not remeasured for this repair. Energy use, archive scale and broader hardware performance remain unestablished.

5.1.7 Validity threats and error analysis

Construct validity is limited by alias matching. An expected value can appear in a wrong explanation, while a valid unlisted paraphrase can be missed. The historical U003 date-role error illustrates why source presence and valid citation indices do not prove claim entailment. Repairing context fitting does not remove these semantic limitations.

Internal validity is limited by reused questions, different schema overheads and unseeded generation at temperature 0.25. The production replay follows an observed failure and prior diagnostic on the same examples. It verifies execution of the applied repair without fitter substitution, but it is not held-out confirmation or a randomized comparison of reasoning ability.

External validity remains narrow: 39 synthetic English Markdown files are not a representative hard drive. No new OCR, multilingual, adversarial, multi-user, longitudinal-memory or Windows 11 GPU study accompanies the repair. Historical summary and resource measurements were not repeated during the question-answer replay.

The strongest supported conclusion is that exact serialization prevents the observed avoidable evidence drop in the tested requests. Sixty-four retained reference messages do not imply 64 correct answers or universally complete context. Frozen baseline records, separate production outputs and source hashes preserve the distinction between original failure, diagnostic development and current local code.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

LocalMind integrates approved-file indexing, source inspection, persistent SQLite memory, project workspaces, local inference and a genuine MCP service. The current Linux and Windows copies include an exact serialized UTF-8 context-fitting repair. The code is applied locally and verified by source hash; it has not been committed or pushed into the historical Git references.

The original generation baseline completed 0/24 positives per method after dropping evidence. The current production replay retains references for 64/64 requests, respects the configured input bound and reports no errors. Single-pass AFVM is 7/24 with macro FVM 37.50%; MCP-agent AFVM is 3/24 with macro FVM 27.08%. These totals match the earlier research-only diagnostic but come from actual repaired production code.

The repair completes one necessary engineering correction without establishing dependable factual assistance. Passage-prefix losses remain, no additional model-selected tool executes and the reused corpus is not independent confirmation. The earlier summary experiment retained only 10/24 requested facts and was not rerun here. Local ownership and functional integration are demonstrated; broader factual reliability, stronger evidence selection and adversarial security remain open work.

6.2 Future scope

Exact serialized-cost fitting is now implemented and regression-tested in both local platform copies. Future context work should preserve per-source evidence quotas, expose the precise references sent to the model and handle requested facts beyond the first 1,000 characters. These are additional improvements, not descriptions of the completed repair. A held-out corpus is needed before claiming general factual reliability.

Task-aware evidence selection remains a priority. Offset-aware retrieval can seek requested fields instead of relying on passage beginnings. Structured extraction and source-linked verification can reduce losses during summary merging. Comparisons of models and agent policies should hold evidence budgets constant, include tasks requiring additional reads and report whether model-selected tools actually retrieve missing facts.

Operational improvements include optional local OCR, opt-in file watching, content-hash freshness, memory export and import, encrypted storage and systematic prompt-injection tests. The tested local repair also needs a versioned commit and fresh native Windows validation before old CI evidence can be attributed to the changed code.

Real Windows 11 GPU runs, heterogeneous and multilingual files, larger archives and consented user studies remain necessary. Future evaluation should measure cold starts, power and human verification effort. These activities are proposed work, distinct from the completed production context repair and its limited replay.

REFERENCES

Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759). https://doi.org/10.1145/1571941.1572114

Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 150–158). https://doi.org/10.18653/v1/2024.eacl-demo.16

Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6465–6488). https://doi.org/10.18653/v1/2023.emnlp-main.398

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). *Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2302.12173

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 6769–6781). https://doi.org/10.18653/v1/2020.emnlp-main.550

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. https://doi.org/10.1145/3359591.3359737

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*. https://arxiv.org/abs/2005.11401

REFERENCES — continued

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Model Context Protocol contributors. (2025). *Model Context Protocol specification* (Revision 2025-11-25). Retrieved October 17, 2026, from https://modelcontextprotocol.io/specification/2025-11-25

Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics* (pp. 2014–2037). https://doi.org/10.18653/v1/2023.eacl-main.148

Ollama. (n.d.). *FAQ*. Retrieved October 17, 2026, from https://docs.ollama.com/faq

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as operating systems* [Preprint, revised 2024]. arXiv. https://doi.org/10.48550/arXiv.2310.08560

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). *Generative agents: Interactive simulacra of human behavior* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2304.03442

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* (pp. 3982–3992). https://doi.org/10.18653/v1/D19-1410

REFERENCES — continued

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval, 3*(4), 333–389. https://doi.org/10.1561/1500000019

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2302.04761

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2303.11366

SQLite developers. (n.d.). *SQLite FTS5 extension*. Retrieved October 17, 2026, from https://sqlite.org/fts5.html

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, 1*. https://arxiv.org/abs/2104.08663

TrijalBurman. (2026a). *LocalMind local knowledge copilot—Linux edition* (Commit ddbbbb7be35a83c6ebc32706bb0aaf41ff379ff8) [Source code]. GitHub. https://github.com/TrijalBurman/MCP-Server/tree/ddbbbb7be35a83c6ebc32706bb0aaf41ff379ff8

REFERENCES — continued

TrijalBurman. (2026b). *Native Windows validation workflow* (Run 37470160711, commit 0e0bbf6f944bc6f2534f882c858ca4cdd22e6443) [Continuous integration execution]. GitHub Actions. https://github.com/TrijalBurman/MCP-Server/actions/runs/37470160711

Vera, H. S., Dua, S., Zhang, B., Salz, D., Mullins, R., Panyam, S. R., Smoot, S., Naim, I., Zou, J., Chen, F., Cer, D., Lisak, A., Choi, M., Gonzalez, L., Sanseviero, O., Cameron, G., Ballantyne, I., Black, K., Chen, K., … Seyedhosseini, M. (2025). *EmbeddingGemma: Powerful and lightweight text representations* [Technical report]. arXiv. https://doi.org/10.48550/arXiv.2509.20354

Wang, W., Dong, L., Cheng, H., Liu, X., Yan, X., Gao, J., & Wei, F. (2023). Augmenting language models with long-term memory. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2306.07174

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., … Qiu, Z. (2025). *Qwen3 technical report* [Technical report]. arXiv. https://doi.org/10.48550/arXiv.2505.09388

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(17), 19724–19731. https://doi.org/10.1609/aaai.v38i17.29946

Appendices

Technical and Submission Records

Appendix A Reproducibility record

The archive preserves the frozen corpus, baseline and diagnostic. The separate production replay is recorded under research/evaluation/results/2026-10-17-production-context-fix-v1/. Verification files in output/verification identify actual production fitting and real MCP calls on the same 32 questions per method.

Historical Linux commit ddbbbb7 and Windows commit 0e0bbf6 precede the local uncommitted repair. Both repaired copies of copilot/ollama.py have SHA-256: 1db1e79b7eac3dba77b2afaa2e852db847b7ae2169d7f92327b750dbaa4cdc25. Full commit identifiers and regression records accompany the archive.

Table A.1 Reproduction settings

Item	Recorded setting
Chat model	qwen3:4b-instruct-2507-q4_K_M
Embedding model	embeddinggemma:latest; 768 dimensions
Chat context / generation	8,192 / 1,536 tokens; temperature 0.25
Runtime	Python 3.14.4; Ollama 0.35.0
Hardware	Ryzen 7 250; RTX 5050 Laptop; nominal 16 GB RAM

Preserve model digests, labels and source hashes; use a new run directory for changes. Restart to load the repair; no model change or reindexing is required. Historical Windows CI run 37470160711 applies to its original commit.

Appendix B Demonstration acceptance cases

Use the supplied examples/knowledge folder for a compact demonstration. Connect it under Library, confirm two documents are indexed and open lighthouse-project.md. The reader should display the complete extracted text; this is the ground truth against which a generated answer is checked. The original files remain in the demonstration folder.

A suitable factual question is: What storage does the Lighthouse project use? The source states SQLite for extracted text, chunks, conversations and memories. A second question asks whether image-only PDFs work: the supplied note says scanned PDFs need OCR, which the application does not provide. Do not substitute a speculative answer if the model abstains.

Table B.1 Demonstration checks and recovery

Check	Expected observation / recovery
Library reader	Source text opens; show it if inference stalls
Project creation	Named workspace appears in Projects
Explicit memory	Saved project fact survives navigation / new chat
Source-linked answer	Compare response with source reader
Optional summary	Verify draft; use saved rehearsal if slow

The normal presentation launcher remains unchanged. Restart any application process left running before the context repair so it imports the corrected code. Verify every answer against the reader, and label any saved screenshot or output as rehearsal evidence. A two-minute live segment should show a few inspectable operations rather than attempt to index an entire drive.

Appendix C Academic integrity and required reports

The supplied format requires an AIC form, a similarity report and an AI plagiarism report. Those institutional records have not been supplied for this submission package. The table below records their status; it is not a replacement certificate or an invented score. Obtain the institution's current forms and reports, review them with the guide and attach them before formal submission.

AI assistance was used for software development, synthetic corpus construction, evaluation support and manuscript/report drafting. Authors remain responsible for checking implementation claims, numerical results, references, institutional declarations and presentation statements. AI assistance does not constitute independent peer review. The synthetic dataset is an explicit experimental artifact rather than naturally collected user behaviour.

Table C.1 Mandatory attachment register

Required attachment	Current status
AIC form	Institution-issued form not provided
Similarity report	Official report and score not provided
AI plagiarism report	Official report and score not provided
Faculty signatures	Unsigned; to be completed by authorized faculty
Student declaration	For authors' review and signature

No signature, approval date, originality percentage or plagiarism outcome has been generated. If institutional policy requires additional disclosure language, the authors should replace this statement with the approved wording after reviewing the underlying evidence.

Appendix D Submission and evidence checklist

The companion IEEE-format research paper acknowledges Dr. Sonali Kothari and lists the four student authors. It is a prepared manuscript; no acceptance, publication, DOI or conference presentation is claimed. The template's optional published-paper page should be attached only after an actual publication exists. Industry, competition and achievement certificates are likewise optional evidence, not created records.

Before printing, verify all names and PRNs, sign the declaration after reading it, obtain authorized faculty signatures and enter the submission date. The report uses the user-supplied department Minors in AIML and academic year 23–27. Keep the editable file and its verified 60-page PDF together so that any later edit can be repaginated and the contents list updated.

Table D.1 Project evidence and preservation

Evidence	Location / handling
Implementation	Historical commits plus current local repair SHA-256
Research evidence	research/evaluation and accompanying archive
Paper	IEEE source and compiled PDF
Presentation preparation	Four-member 10-minute script and demo guide
Local data	Back up SQLite and project PLAN.md with app stopped

This updated report separates the historical baseline, earlier diagnostic and current local production repair. The new replay and exact source hashes accompany the archive; the repair remains uncommitted. Future changes must retain their own version and verification records so that prior findings remain reproducible.